CITY CARE

Documentación completa de consulta y seguimiento

Etapa 3 3 del módulo de reportes e incidencias

La etapa 3.3 completa la consulta y el seguimiento de los reportes. El ciudadano puede listar y revisar únicamente sus incidencias; el personal administrativo consulta el ámbito que le corresponde y cambia el estado mediante transiciones controladas. Cada movimiento queda registrado para conservar trazabilidad.

Indicador	Definición
Estado	Completo y validado
Rutas nuevas	5 operaciones de consulta y seguimiento
Tabla nueva	report_status_histories
Migraciones totales	12 aplicadas en MySQL
Endpoints totales	22 rutas registradas
Suite funcional	57 de 57 pruebas aprobadas
Contrato	OpenAPI y cliente Orval sincronizados

Resultado de la etapa

El backend ofrece listados paginados, detalle con evidencia e historial, filtros por criterios operativos y cambio de estado con permisos y alcance por dependencia. La creación del reporte conserva el estado inicial pending y registra de forma transaccional el primer movimiento del historial.

1 Objetivo y alcance

- Permitir que cada ciudadano consulte solamente los reportes creados con su cuenta.
- Entregar listados paginados con búsqueda y filtros reutilizables desde Flutter o un panel web.
- Mostrar el detalle del reporte con ubicación, categoría, fotografía firmada y trazabilidad.
- Permitir seguimiento global al administrador general y seguimiento local al administrador de dependencia.
- Controlar las transiciones entre pending, in_progress y resolved.
- Exigir una observación cuando un reporte retrocede o se reabre.
- Registrar actor, estado anterior, estado nuevo, origen, observación y fecha de cada movimiento.
- Actualizar el contrato OpenAPI y regenerar el cliente TypeScript mediante Orval.
- Comprobar permisos, aislamiento, filtros, paginación e historial mediante pruebas funcionales.

2 Arquitectura funcional

Componente	Responsabilidad
Flutter o frontend	Solicita listados, detalle y actualización visual de estados.
Middleware auth	Identifica al usuario exclusivamente mediante el token Bearer.
Middleware permission	Autoriza reports.list, reports.read o reports.status.
ReportQueriesController	Aplica filtros, alcance, paginación y transiciones.
ReportPresenterService	Genera la respuesta pública y la URL temporal de MinIO.
MySQL	Conserva reportes y el historial permanente de estados.
MinIO	Entrega la imagen por una URL firmada que no se almacena en la base.
OpenAPI y Orval	Publican las operaciones y funciones consumibles por el frontend.

La consulta de datos y la consulta de imágenes están separadas. MySQL devuelve la referencia de la evidencia; el presentador solicita a MinIO una URL nueva para cada respuesta. Esta URL expira y no debe persistirse en Flutter.

3 Modelo de datos

La etapa agrega report_status_histories. No duplica los datos principales del reporte y no modifica la imagen. La relación es de uno a muchos: un reporte puede acumular todos los cambios necesarios durante su atención.

Campo	Tipo	Uso
id	entero	Identificador interno del movimiento.
report_id	entero FK	Reporte relacionado; la eliminación en cascada conserva integridad.
previous_status	texto nullable	Estado anterior; es nulo en el primer registro.
new_status	texto	Estado que quedó vigente después del movimiento.
changed_by_user_id	entero FK nullable	Usuario que realizó el cambio; SET NULL protege el historial.
source	texto	creation, admin o migration.
note	texto nullable	Observación operativa o motivo del movimiento.
created_at	fecha y hora	Momento exacto del cambio.

4 Migraciones

Migración	Resultado
1796000000300 create report status histories table	Crea la tabla, llaves foráneas e índices por reporte, actor y estado.
1796000000400 backfill report status histories	Añade un registro inicial a los reportes que existían antes del 3.3.

Los reportes nuevos crean su historial dentro de la misma transacción que reports y report_evidences. Si una parte falla, no queda un reporte sin evidencia o sin estado inicial.

5 Flujo del ciudadano

1. El ciudadano inicia sesión y obtiene un token Bearer.
2. La aplicación solicita GET /api/v1/reports con los filtros necesarios.
3. El backend toma user_id desde el token y fuerza el listado a ese usuario.
4. La consulta aplica búsqueda, categoría, estado, fechas, página y tamaño de página.
5. Cada elemento recibe una fotoUrl firmada y temporal.
6. Al abrir un elemento, GET /api/v1/reports/:id exige que el reporte pertenezca al ciudadano.
7. El detalle devuelve el historial completo en orden cronológico.
8. Un reporte ajeno responde 404 para no confirmar que el recurso existe.

6 Flujo administrativo

1. El middleware verifica reports.list, reports.read o reports.status según la operación.
2. El administrador general puede consultar cualquier dependencia.
3. El administrador de dependencia queda limitado a actor.dependency_id.
4. El listado puede filtrar también por dependencia y ciudadano cuando el alcance lo permite.
5. El detalle incluye datos mínimos del ciudadano y el historial.
6. Antes de cambiar un estado se valida el alcance, la transición y la observación requerida.
7. La actualización del reporte y la inserción del historial se confirman en una transacción.
8. El movimiento se registra además en los logs con actor, reporte y estados anterior y nuevo.

7 Endpoints del punto 3 3

Método	Ruta	Acceso	Función
GET	/api/v1/reports	Ciudadano autenticado	Lista solamente sus reportes.
GET	/api/v1/reports/:id	Propietario	Detalle propio con historial.
GET	/api/v1/admin/reports	reports.list	Listado administrativo por alcance.
GET	/api/v1/admin/reports/:id	reports.read	Detalle para seguimiento.
PATCH	/api/v1/admin/reports/:id/status	reports.status	Cambio controlado de estado.

8 Filtros y paginación

Parámetro	Disponible	Regla
page	Todos los listados	Entero positivo; valor predeterminado 1.
perPage	Todos los listados	De 1 a 100; valor predeterminado 20.
search	Todos los listados	Busca en folio, descripción y dirección.
status	Todos los listados	pending, in_progress o resolved.
categoryId	Todos los listados	Identificador positivo de categoría.
from	Todos los listados	Fecha inicial inclusiva en formato YYYY-MM-DD.
to	Todos los listados	Fecha final inclusiva en formato YYYY-MM-DD.
dependencyId	Administración	Global; un administrador local no puede salir de su ámbito.
userId	Administración	Filtra por ciudadano.

La fecha inicial no puede ser posterior a la final. Los valores de fecha inexistentes y los estados fuera del catálogo producen una respuesta 422 uniforme.

9 Estados y transiciones

Estado actual	Estado permitido	Observación	Significado
pending	in_progress	Opcional	La dependencia comienza la atención.
in_progress	resolved	Opcional	La incidencia fue atendida.
in_progress	pending	Obligatoria	La atención se devuelve a la cola.
resolved	in_progress	Obligatoria	El reporte se reabre.

No se permite pending a resolved directamente. Tampoco se acepta guardar el mismo estado nuevamente. La matriz obliga a que el ciclo operativo sea explícito y evita movimientos sin trazabilidad.

10 Petición de cambio de estado

```
PATCH /api/v1/admin/reports/42/status
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "in_progress",
  "note": "La cuadrilla inició la revisión"
}
```

status es obligatorio. note admite de 3 a 1000 caracteres y se vuelve obligatorio al regresar de in_progress a pending o al reabrir de resolved a in_progress.

11 Historial entregado al frontend

```
"historial": [
  {
    "id": 1,
    "estadoNuevoCodigo": "pending",
    "estadoNuevo": "Pendiente",
    "origen": "creation",
    "observacion": "Reporte creado por el ciudadano",
    "cambiadoPor": { "id": 15, "nombre": "Ana López" },
    "creadoEn": "2026-11-01T07:30:00.000Z"
  }
]
```

12 Estructura de las respuestas

Bloque	Contenido
Identidad	id y folio.
Categoría	categoriaId, categoriaCodigo y categoria.
Ubicación	dirección opcional, latitud, longitud, precisión y origen.
Seguimiento	estadoCodigo, estado y fechas de creación y actualización.
Evidencia	fotoUrl temporal, fotoUrlExpira y estadoEvidencia.
Dependencia	Identificador y nombre cuando existe asignación.
Ciudadano	Solo en respuestas administrativas.
Historial	Solo en detalle y después de un cambio de estado.
Paginación	data y meta en los listados.

13 Ejemplo de listado

```
{
  "data": [{
    "id": 42, "folio": "CC-2026-000042",
    "categoria": "Bache", "estadoCodigo": "pending",
    "estado": "Pendiente", "fotoUrl": "http://10.0.2.2:9000/..."
  }],
  "meta": { "total": 1, "perPage": 20, "currentPage": 1, "lastPage": 1 }
}
```

fotoUrl debe utilizarse de inmediato. Cuando expire, la aplicación vuelve a consultar el listado o el detalle para recibir una firma nueva.

14 Permisos y alcance

Rol	Reportes propios	Reportes administrativos	Cambio de estado
user	Sí	No	No
dependency_admin	Sí	Solo su dependencia	Solo su dependencia
admin	Sí	Todas las dependencias	Todas las dependencias

Los permisos agregados son reports.list, reports.read y reports.status. El middleware comprueba el permiso, pero el controlador también verifica dependency_id; poseer el permiso no permite consultar otra dependencia.

15 Errores controlados

HTTP	Código	Causa
401	E_UNAUTHORIZED_ACCESS	Token ausente o inválido.
403	E_PERMISSION_REQUIRED	El rol no posee la operación administrativa.
403	E_REPORT_SCOPE	Reporte fuera de la dependencia permitida.
404	E_REPORT_NOT_FOUND	Reporte inexistente o ajeno al ciudadano.
409	E_REPORT_STATUS_TRANSITION_INVALID	Movimiento no permitido por la matriz.
422	E_REPORT_STATUS_INVALID	Estado fuera del catálogo.
422	E_REPORT_STATUS_NOTE_REQUIRED	Falta observación para retroceder o reabrir.
422	E_REPORT_DATE_RANGE_INVALID	Fecha inválida o rango invertido.
422	E_VALIDATION_ERROR	Paginación o parámetros con formato incorrecto.

16 OpenAPI y Orval

El contrato versionado openapi/city-care.v1.yaml contiene las cinco operaciones nuevas, parámetros de consulta, cuerpos, respuestas y esquemas. Orval validó el documento y regeneró sdk/generated/city-care.ts.

Operación OpenAPI	Función Orval	Uso
listMyReports	listMyReports	Listado del ciudadano.
getMyReport	getMyReport	Detalle del ciudadano.
listAdminReports	listAdminReports	Listado administrativo.
getAdminReport	getAdminReport	Detalle administrativo.
updateReportStatus	updateReportStatus	Actualización de estado.

El SDK generado no debe editarse manualmente. Cuando cambie un controlador o una interfaz, se actualiza OpenAPI y después se ejecuta npm run openapi:generate.

17 Componentes implementados

Archivo	Responsabilidad
report_queries_controller.ts	Listados, detalle, filtros, alcance y cambio de estado.
report_presenter_service.ts	Respuesta pública, historial y URL firmada.
report_status_history.ts	Modelo y relaciones del historial.
report.ts y user.ts	Relaciones con movimientos de estado.
report.ts validator	Paginación, filtros y cuerpo de actualización.
authorization_service.ts	Permisos reports.list, reports.read y reports.status.
reports_controller.ts	Inserta el estado inicial al crear el reporte.
start/routes.ts	Registra las cinco rutas nuevas.
handler.ts	Mensajes uniformes para errores del seguimiento.

18 Pruebas y evidencia de validación

Comprobación	Resultado
Aislamiento ciudadano	No puede consultar reportes de otra cuenta.
Paginación y filtros	Búsqueda, estado, categoría y límites aprobados.
Alcance administrativo	dependency_admin queda restringido a su dependencia.
Administrador global	Puede consultar reportes de varias dependencias.
Transiciones	Avance, retroceso, resolución y reapertura validados.
Historial	Creación y cada cambio conservan trazabilidad.
Errores	Estados y rangos de fechas inválidos responden 422.
Suite completa	57 de 57 pruebas aprobadas.
TypeScript	npm run typecheck sin errores.
ESLint	npm run lint sin errores.
Build	Compilación de producción correcta.
OpenAPI y Orval	Contrato válido y SDK regenerado.
MySQL	12 migraciones completed y 9 tablas de aplicación.
Rutas	22 endpoints registrados.

19 Guía de comprobación manual

1. Iniciar MySQL, MinIO y el backend.
2. Iniciar sesión con una cuenta que tenga reportes.
3. Ejecutar GET /api/v1/reports y confirmar data, meta y fotoUrl.
4. Abrir GET /api/v1/reports/:id y revisar historial en orden cronológico.
5. Iniciar sesión como dependency_admin y listar /api/v1/admin/reports.
6. Confirmar que el listado no muestre otra dependencia.
7. Cambiar pending a in_progress mediante PATCH.
8. Cambiar in_progress a resolved y comprobar el nuevo movimiento.
9. Reabrir resolved a in_progress con una observación.
10. Actualizar el detalle y confirmar que la foto siga disponible con una URL nueva.

20 Diagnóstico de problemas

Síntoma	Causa probable	Revisión
Lista vacía	La cuenta no creó reportes o los filtros no coinciden.	Quitar filtros y verificar token.
404 en detalle propio	El reporte pertenece a otra cuenta.	Comparar usuario del token.
403 administrativo	Falta permiso o dependencia fuera de alcance.	Rol, permisos y dependency_id.
409 al cambiar estado	La transición no está permitida.	Consultar la matriz del punto 9.
422 al reabrir	Falta note o tiene menos de 3 caracteres.	Enviar una observación útil.
422 en fechas	Formato incorrecto o rango invertido.	Usar YYYY-MM-DD y from menor o igual a to.
La foto no abre	La URL firmada expiró.	Consultar de nuevo el listado o detalle.
Historial vacío	Migración o inserción inicial pendiente.	Revisar migration:status y la transacción.

21 Seguridad y decisiones técnicas

- El user_id de la consulta ciudadana siempre proviene del token.
- Un recurso ajeno responde 404 al ciudadano para reducir exposición de información.
- El permiso administrativo se combina con una comprobación de dependencia.
- Los cambios de estado y el historial se guardan en una sola transacción.
- La eliminación de un usuario conserva el historial mediante changed_by_user_id nullable.
- Las fotografías siguen privadas; solamente se entregan URLs temporales.
- La observación de retrocesos y reaperturas evita cambios operativos sin explicación.
- Los logs registran identificadores y estados, pero no tokens ni credenciales.

22 Cierre de la etapa

La etapa 3.3 queda concluida. El ciudadano dispone de Mis reportes y detalle; los administradores disponen de seguimiento con alcance; los estados siguen una matriz explícita; y cada movimiento queda respaldado por un historial consultable. La base está preparada para el punto 3.4, donde se incorporarán equipos de trabajo, asignación, prioridad, fechas y observaciones operativas.